

Red Hat AI Training: Models-as-a-Service (MaaS)

SMEs for development:

- Taylor Smith
- James Harmison
- Jonathan Zarecki (PM - consulted only)

Problem Statement

Organizations currently face a "GPU Silo" crisis where expensive, limited hardware is trapped within individual teams, leading to simultaneous idle capacity and project bottlenecks. Without a centralized Models as a Service (MaaS) layer, enterprises lack the orchestration needed to enforce usage quotas, ensure multi-tenant security, and provide a standardized model library. This fragmented approach results in redundant model deployments, wasted hardware, and prevents the delivery of scalable AI solutions. MaaS solves this by acting as a governance layer sitting directly between users and the model serving infrastructure, maximizing the ROI of scarce hardware.

Course Goal

To equip architects, engineers, and administrators with the knowledge and skills required to design, configure, deploy, and monitor a robust Models as a Service (MaaS) solution using Red Hat OpenShift AI 3.4. By the end of this training, participants will understand the strict boundaries around serving, resource management, access, and lifecycle. You will learn to govern a scalable MaaS ecosystem, transforming isolated GPUs into a fairly scheduled, multi-tenant shared service.

Major Changes in this Release

- **Platform Update & Tech Preview Status:** Updating the platform to Red Hat OpenShift AI 3.4, where Models-as-a-Service (MaaS) is introduced as a Technology Preview feature.
- **CRD Configuration Shift:** Focuses on activating the MaaS feature by setting the `modelsAsService` component to `Managed` within the `DataScienceCluster` CRD.
- **Underlying Architecture & Routing:** Heavy emphasis on Kuadrant (Authorino/Limitador), KServe, and the new Gateway API (utilizing the `maas-default-gateway` for routing).
- **Llama Stack Operator Dependency:** Accessing the MaaS dashboard interface now strictly requires the Llama Stack Operator to be installed and enabled on the cluster.
- **Gen AI Studio & Dashboard UI:** The MaaS developer interface is now located within the "Gen AI studio" menu. Platform engineers must explicitly enable this by setting both `genAiStudio: true` and `modelAsService: true` in the `OdhDashboardConfig`.

- **AI Available Assets Page:** Models published as MaaS endpoints are now globally visible and consumable directly from the new AI Available Assets catalog, enabling centralized discovery across all projects.

Target Audience

- **Platform Engineers:** Responsible for designing and deploying model serving environments, enabling MaaS, defining service tiers, and deploying governed models.
- **Developers:** Responsible for generating API keys, connecting applications to OpenAI-compatible REST endpoints, and integrating code assistants.
- ~~**DevOps / SREs:** Focused on observability, diagnostics, operational scale, tracking consumption metrics, cost allocation, and utilizing agentic AI for cluster operations.~~

Prerequisite Skills/Knowledge

- Familiarity with Kubernetes and Red Hat OpenShift Container Platform.
 - Basic understanding of machine learning concepts and Red Hat OpenShift AI.
 - Basic networking, routing, and understanding of Kubernetes operators.
 - Familiarity with REST APIs and Prometheus/Grafana observability.
-

Performance Objectives (POs)

- **[PO 1] Design a MaaS Architecture:** Understand the "GPU Silo" crisis and map the architectural layers. This includes understanding the roles of KServe, Kuadrant, and the Gateway API to establish strict serving boundaries, as well as mapping the foundational GPU strategy using the NVIDIA GPU Operator stack.
- **[PO 2] Configure the MaaS Platform & Ecosystem:** Enable MaaS by updating the DataScienceCluster CR to Managed and install necessary ecosystem operators (NFD, Cert-Manager, Connectivity link, Kueue (RHBoK), Leader Worker Set) to manage GPUs as shared resources and ensure fair scheduling. Configure access controls and rate limits for default tiers (Free, Premium, Enterprise), and demonstrate the ability to create and apply custom service tiers.
- **[PO 3] Formulate a Model Selection Strategy:** Evaluate models based on architecture (dense vs. sparse), capabilities (reasoning vs. non-reasoning), context length, and tool calling considerations to align with specific customer workloads and hardware impacts.
- **[PO 4] Deploy Models for MaaS:** Deploy a model (e.g., Nemotron 3 Nano 30B A3B) transitioning from single-node vLLM to the distributed llm-d runtime and publish it as a

MaaS endpoint to authorized tiers. Proper instrument model endpoints, manage model versions, and integrate the deployment into GitOps pipelines.

- **[PO 5] Track Metrics and Usage:** Utilize the in-cluster Prometheus instance and Grafana to visualize token counts, request rates, latency metrics (TTFT, TPOT), and response outcomes. Connect hardware-level data via the NVIDIA DCGM Exporter to platform metrics to establish cross-department chargeback models.
- **[PO 6] Consume MaaS Endpoints & Agentic Integrations:** Navigate the OpenShift AI dashboard to generate API keys and consume models using OpenAI-compatible REST API endpoints. Connect the published model endpoint to downstream applications, such as OpenShift DevSpaces, to act as an IDE code assistant.

Considerations and Risks

- **Lab Environment Dependencies:** This course requires significant hardware acceleration (GPUs) in the lab environment. Cluster environments must be capable of supporting distributed inference (llm-d), meaning multi-GPU node availability is required for the deployment labs.
- **Technical Preview Status:** Note that MaaS is currently a Technical Preview Feature in version 3.4, meant for early access and testing, with no SLAs or official support yet.

Products/Technologies

- Red Hat OpenShift Container Platform (4.20+).
- Red Hat OpenShift AI 3.4.
- Red Hat Connectivity Link (Kuadrant: Authorino, Limitador).
- KServe & Gateway API.
- llm-d and vLLM serving runtimes.
- Ecosystem Operators (NFD, Cert-Manager, Kueue, Leader Worker Set).

Progressive Diagram Build Strategy

Introduce a layered architecture diagram, building it from the bottom up as students progress through the modules:

- **Lessons 1 & 2 (The Base):** Foundational Accelerator Enablement and Ecosystem Operators.
- **Lessons 3 & 4 (The Core):** LLM Inference, model selection, and deployment.
- **Lessons 5 & 6 (The Capstone):** MaaS governance, observability, chargeback, and developer consumption.

HIGH LEVEL DESIGN (HLD)

Lesson 1: Designing a Models as a Service Architecture

Lesson Goal: Understand the business value of MaaS and map the core network and serving architecture.

- **Section 1.1: Solving the GPU Silo Crisis:** Explain how organizations face a crisis where expensive hardware is trapped within individual teams. Discuss how MaaS acts as a centralized governance layer to optimize hardware utilization and deliver AI models as shared multi-tenant resources.
- **Section 1.2: The Core Architecture Stack & GPU Strategy [PO1]:** Map the architectural layers, specifically understanding the roles of KServe, Kuadrant, and the Gateway API. **[Diagram Build - Layer 1]:** Introduce the "Foundational Accelerator Enablement" layer at the hardware level, detailing the NVIDIA GPU Operator, including drivers, DCGM, the container toolkit, MIG, and the manager.
- **Section 1.3: Architecture Validation:** Test recall of architectural layers and components through a knowledge check.

Lesson 2: Configuring the MaaS Platform and Ecosystem

Lesson Goal: Prepare the OpenShift cluster, install foundational ecosystem operators, enable the MaaS functionality within OpenShift AI, and establish governance tiers.

- **Section 2.1: Hardware Acceleration and Ecosystem Operators:** Install Node Feature Discovery (NFD) for hybrid cloud GPU recognition. Deploy Cert-Manager, Kueue, and the Leader Worker Set to manage workload queuing across the foundational GPU layer.
- **Section 2.2: Enabling the MaaS Component:** Update the modelsAsService component in the DataScienceCluster CR to Managed to trigger the instantiation of MaaS features.
- **Section 2.3: Defining Governance Tiers:** Integrate with Kubernetes RBAC to map users to service tiers based on OpenShift groups. Configure access controls and rate limits for default tiers like Level 0 (Free), Level 1 (Premium), and Level 2 (Enterprise), while also demonstrating how to create **custom service tiers**.

Lesson 3: Model Selection Strategy (Reintegrated Requirement)

Lesson Goal: Gain the technical expertise to select optimal model architectures based on customer needs and hardware impact.

- **Section 3.1: Dense vs. Sparse Models:** Understand the architectural differences and performance impacts of choosing dense versus sparse models.
- **Section 3.2: Capability Assessment:** Evaluate models based on reasoning vs. non-reasoning capabilities.
- **Section 3.3: Context & Tooling:** Understand context length limitations and tool calling considerations when choosing the right model type for specific workloads.

Lesson 4: Deploying a Model for MaaS

Lesson Goal: Deploy a large language model transitioning from single-node to distributed inference, manage versions, and expose it securely to the MaaS ecosystem.

- **Section 4.1: Distributed Inference and llm-d:** Understand why llm-d is required for MaaS deployments to optimize disaggregated serving and prefix cache aware routing. Review how vLLM manages GPU memory via page attention. **[Diagram Build - Layer 2]:** Show how llm-d and vLLM sit on top of the GPU foundation to maximize Tokens/GPU.
- **Section 4.2: Publishing and Instrumenting:** Deploy a model (using the NVIDIA-Nemotron-3-Nano-4B-FP8) as the standard lab model) and select "Publish as MaaS endpoint". Learn how to properly **instrument model endpoints** and enforce access controls.
- **Section 4.3: Lifecycle Management:** Manage **model versions** and integrate the model serving deployment into existing **GitOps pipelines**.

Lesson 5: Tracking Metrics and Usage

Lesson Goal: Monitor operational metrics for capacity planning, cost allocation, and performance diagnostics.

- **Section 5.1: The MaaS Observability Stack:** Understand how OpenShift AI exposes centralized metrics via User Workload Monitoring. **[Diagram Build - Layer 3]:** Connect the NVIDIA DCGM Exporter to the platform metrics to demonstrate how hardware data flows up to the observability plane. Discuss how this data enables cross-department **chargeback** models.
- **Section 5.2: Visualizing Token and Request Data:** Access Grafana to build queries tracking rate limit hits, token counts, request success, and latency-focused metrics like Time To First Token (TTFT) and Time Per Output Token (TPOT).

Lesson 6: API Integration for Developers

Lesson Goal: Enable downstream applications to securely consume deployed models and monitor token usage.

- **Section 6.1: The Developer Experience:** Navigate the "Models as a service" tab in the OpenShift AI dashboard to generate single-view API keys. Explain the OpenAI-compatible API and standardized error codes like 401, 403, and 429.
- **Section 6.2: Consuming the OpenAI-Compatible API:** Use curl/Python to interact with the `/v1/chat/completions` endpoint using Bearer token authentication. Read the header to identify remaining tokens and experiment with setting max tokens.

- **Section 6.3: Agentic Integration: [Diagram Build - Final Capstone]:** Connect the published Nemotron model to **OpenShift DevSpaces** acting as an IDE with a code assistant, utilizing standard REST API calls and the generated MaaS endpoint.